

BST

Advanced Operations — LCA, Height, Counting & Validation

Topic

This lab extends your BST implementation with advanced recursive operations. You will implement and explore: counting nodes and leaves, computing tree height, finding minimum and maximum values, mirroring the tree, validating BST correctness, and finding the Lowest Common Ancestor (LCA) of two nodes.

Objective

- Implement recursive functions to count total and leaf nodes.
- Compute the height of a BST recursively.
- Find minimum and maximum values using BST properties iteratively.
- Mirror a BST by swapping left and right children at every node.
- Validate whether a binary tree satisfies BST properties.
- Understand and implement Lowest Common Ancestor (LCA) using BST properties.

Outcomes

- Count all nodes and leaf nodes in a BST recursively.
 - Determine the height of a BST and relate it to tree balance.
 - Navigate directly to minimum and maximum values using BST ordering.
 - Apply tree mirroring and observe its effect on traversal order.
 - Detect invalid BSTs by checking value ranges at each node.
 - Explain and implement LCA using BST structural properties.
-

Concept: Lowest Common Ancestor (LCA)

What is LCA?

The Lowest Common Ancestor of two nodes is the deepest node in the tree that has both nodes as descendants. A node is considered a descendant of itself.

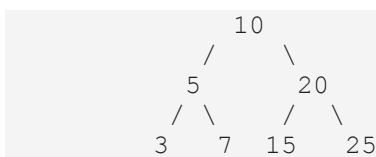
Think of it like a family tree — the LCA is the closest shared ancestor of two people. Both a grandfather and a father may be common ancestors, but the father is the LOWEST (deepest) one.

Key Terms

Term	Meaning
Ancestor	Any node on the path from the root down to a given node
Common Ancestor	A node that is an ancestor of both val1 and val2
Lowest Common Ancestor	The deepest (furthest from root) common ancestor
Descendant of itself	A node counts as its own ancestor

Example Tree

Build this BST by inserting: 10, 5, 20, 3, 7, 15, 25



Example 1 — LCA(3, 7)

Question: What is the lowest common ancestor of nodes 3 and 7?

Step	Current Node	3 and 7 vs node?	Decision
1	10	Both $3 < 10$ and $7 < 10$	Go LEFT
2	5	$3 < 5$ but $7 > 5$ — split!	Return 5 as LCA

Answer: $LCA(3, 7) = 5$

Why not 10? Because 10 is also a common ancestor but it is shallower (depth 0). Node 5 is deeper (depth 1) and still a common ancestor of both — so 5 is the LOWEST.

Example 2 — LCA(7, 15)

Question: What is the lowest common ancestor of nodes 7 and 15?

Step	Current Node	7 and 15 vs node?	Decision
1	10	$7 < 10$ but $15 > 10$ — split!	Return 10 as LCA

Answer: $LCA(7, 15) = 10$

The values diverge immediately at the root — one goes left, the other goes right — so the root is the LCA.

Example 3 — LCA(5, 3)

Question: What is the lowest common ancestor of nodes 5 and 3?

Step	Current Node	5 and 3 vs node?	Decision
1	10	Both $5 < 10$ and $3 < 10$	Go LEFT
2	5	curr equals val1 (5)	Return 5 as LCA

Answer: $LCA(5, 3) = 5$

A node is its own ancestor. Since 5 equals one of the values, the current node itself is the LCA — no need to go deeper.

The Three Decision Rules

At every node during the LCA search, apply exactly one of these three rules:

Condition	Action	Example
Both values $<$ current node	Go LEFT — LCA is in left subtree	$LCA(3,7)$ at 10 $\rightarrow$ left
Both values $>$ current node	Go RIGHT — LCA is in right subtree	$LCA(15,25)$ at 10 $\rightarrow$ right
Values split OR one equals curr	curr IS the LCA — return it	$LCA(7,15)$ at 10 $\rightarrow$ return 10

Tasks

Task 1: Count Total Nodes

Write a recursive function that returns the total number of nodes in the BST.

```
int count(BNode<T>* curr)
{
    // Base case: empty tree
    // Recursive case: 1 + count(left) + count(right)
}
```

Hint

Every node contributes 1 to the count.

Return 0 for NULL (base case).

Return $1 + \text{count}(\text{left}) + \text{count}(\text{right})$ for any non-NULL node.

Test with insertions: 10, 20, 5, 15, 25 — expected count = 5.

Task 2: Count Leaf Nodes

Write a recursive function that returns the number of leaf nodes (nodes with no children).

```
int count_leaves(BNode<T>* curr)
```

```
{
    // A leaf has left == NULL && right == NULL
    // Return 1 for a leaf, else recurse left + right
}
```

Hint

Check three cases: NULL (return 0), leaf (return 1), internal node (recurse).

A node is a leaf when BOTH left and right pointers are NULL.

For the tree 10, 20, 5, 15, 25 — the leaves are 5, 15, 25 so expected = 3.

Note: you do NOT need to visit NULL children for a leaf — return 1 immediately.

Task 3: Find Height of BST

Write a recursive function that returns the height of the BST (number of edges on the longest root-to-leaf path).

```
int height(BNode<T>* curr)
{
    // Height of NULL tree = -1 (or 0 depending on definition)
    // Height = 1 + max(height(left), height(right))
}
```

Hint

Return -1 for NULL (so a single node has height 0).

Use max() from <algorithm> to pick the taller subtree.

For tree 10, 5, 20, 3, 7 — height = 2 (path: 10 -> 5 -> 3).

A taller tree = more levels = more recursive calls needed to reach a leaf.

Task 4: Find Minimum and Maximum

Write two iterative functions to find the minimum and maximum values in a BST without traversal.

```
T find_min(BNode<T>* curr)
{
    // Keep going LEFT until no left child
    // That node holds the minimum
}

T find_max(BNode<T>* curr)
{
    // Keep going RIGHT until no right child
    // That node holds the maximum
}
```

Hint

BST property guarantees: smallest value is at the leftmost node.

BST property guarantees: largest value is at the rightmost node.

Use a while loop — no recursion needed.

Do NOT traverse the entire tree — use the BST ordering to navigate directly.

Task 5: Mirror / Invert the BST

Write a recursive function that inverts the BST by swapping left and right children at every node.

```
void mirror(BNode<T>* curr)
{
    // Base case: NULL node, return
    // Swap curr->left and curr->right
    // Recurse on both children
}
```

Hint

Use a temp pointer to swap: temp = left; left = right; right = temp.

After mirroring, inorder traversal gives values in DESCENDING order.

Think about this: why does swapping at every node produce the mirror image?

Call mirror() then print inorder — verify the output is reversed.

Task 6: Validate a BST

Write a recursive function that checks whether a binary tree satisfies BST properties at every node.

```
bool is_valid_BST(BNode<T>* curr, T min_val, T max_val)
{
    // curr's data must be strictly > min_val AND < max_val
    // Recurse left with updated max, right with updated min
}
```

Hint

Pass a valid range [min_val, max_val] down to each node.

Left child: new max = curr->data (left subtree must be smaller).

Right child: new min = curr->data (right subtree must be larger).

Call with is_valid_BST(root, INT_MIN, INT_MAX) from outside.

Test by manually constructing an invalid tree and confirming it returns false.

Task 7: Lowest Common Ancestor (LCA)

Using the concept explained above, implement the LCA function for a BST.

```
BNode<T>* LCA(BNode<T>* curr, T val1, T val2)
{
    if (curr == NULL)
```

```

    return NULL;

    // Both values smaller than curr -> LCA is in left subtree
    if (val1 < curr->data && val2 < curr->data)
        return LCA(curr->left, val1, val2);

    // Both values larger than curr -> LCA is in right subtree
    if (val1 > curr->data && val2 > curr->data)
        return LCA(curr->right, val1, val2);

    // Values split here, or one equals curr -> curr IS the LCA
    return curr;
}

```

Hint — LCA Decision Rules

Rule 1: If both val1 and val2 are LESS than curr->data, go LEFT.

Rule 2: If both val1 and val2 are GREATER than curr->data, go RIGHT.

Rule 3: Otherwise (values split, or one equals curr) — return curr as LCA.

Remember: a node is its own ancestor. $LCA(5, 3) = 5$, not 10.

The order of val1 and val2 does not matter — $LCA(3,7) = LCA(7,3) = 5$.

Test your implementation with the following cases:

Function Call	Expected LCA	Reason
$LCA(\text{root}, 3, 7)$	5	3 and 7 both under 5, split at 5
$LCA(\text{root}, 7, 15)$	10	7 goes left, 15 goes right at 10
$LCA(\text{root}, 15, 25)$	20	Both greater than 10, split at 20
$LCA(\text{root}, 5, 3)$	5	5 equals curr — node is its own ancestor
$LCA(\text{root}, 3, 25)$	10	3 goes left, 25 goes right at root

Great Job, Rest Time 

Page 1